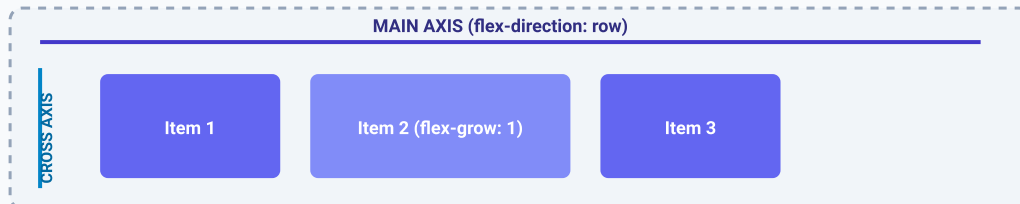


Day 6: Flexbox, Grid, & CSS Positioning

Mastering 1D flex alignment, 2D grid systems, and coordinate positioning models.

1. CSS Flexbox (1-Dimensional Layouts)

Flexbox organizes items along a **single axis** (either a row or a column). It dynamically distributes free space and aligns content across elements of varying sizes.



Flex Container Properties (Parent)

```
.flex-container { display: flex;
flex-direction: row; /* row | column */
justify-content: space-between; align-items: center; gap: 16px; }
```

Flex Item Properties (Child)

```
.flex-item { /* grow | shrink | basis */ flex: 1 0 200px; align-self: flex-end; }
```

`flex-grow: 1` fills available extra space proportionally.

2. CSS Grid (2-Dimensional Layouts)

CSS Grid handles **both rows and columns simultaneously**. It defines explicit track structures that child items align into seamlessly.

Explicit Columns & Gap

```
.grid-container { display: grid;
grid-template-columns: repeat(3, 1fr);
gap: 20px; }
```

The `1fr` unit splits available horizontal space into equal fractional shares.

Responsive Auto-Fit (No Media Queries)

```
.responsive-grid { display: grid;
grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
gap: 16px; }
```

Automatically wraps items when screens drop below 220px.

3. Flexbox vs. CSS Grid Decision Matrix

Feature	Flexbox	CSS Grid
Dimensions	1D (Row or Column)	2D (Rows <i>and</i> Columns together)
Design Strategy	Content-driven (items dictate size)	Layout-driven (grid defines rows & columns)
Item Spanning	Items wrap, but cannot span across rows	Items cleanly span multiple rows or columns
Primary Use Cases	Navbars, button groups, vertical centering	Main page layouts, dashboard cards, photo galleries

4. CSS Positioning (`position`)

The `position` property takes elements out of standard document flow or offsets them relative to reference targets.

Value	Behavior & Flow Context	Common Use Case
<code>static</code>	Default flow. Ignores <code>top</code> , <code>left</code> , and <code>z-index</code> .	Standard block layout.
<code>relative</code>	Offset relative to normal position. Acts as anchor parent.	Parent container for absolute badges.
<code>absolute</code>	Removed from flow. Positioned relative to nearest non-static ancestor.	Card badges, dropdowns, close buttons.
<code>fixed</code>	Removed from flow. Positioned relative to the viewport window .	Sticky navbars, modal backdrops, top alerts.
<code>sticky</code>	Flows naturally until scroll threshold, then sticks in place.	Sticky section titles, table headers.

5. Stacking Order (`z-index`)

Controls vertical overlapping order along the Z-axis. Higher values sit on top of lower values. **Note:** `z-index` only applies to elements with a `position` other than `static`.

6. Component Blueprint: Card with Absolute Badge

```
/* Combined Day 6 Layout Blueprint */
.plant-card { position: relative; /* Anchor for absolute children */
display: flex; /* 1D flex content layout */
flex-direction: column;
padding: 16px; border: 1px solid #cbd5e1; border-radius: 8px; }
.badge-new { position: absolute; /* Positioned relative to .plant-card */
top: 12px; right: 12px; background-color: #4338ca; color: #ffffff; padding: 4px 8px; z-index: 10; /* Stacked above card image */ }
```